# pyDelphin

## recent updates

and

# gTest

## tutorial

Michael Wayne Goodman | https://github.com/goodmami

# pyDelphin improvements overview:

- SimpleMrs 1.1
- delphin.mrs.path
- delphin.mrs.compare.isomorphic()
- delphin.itsdb.make_skeleton()
- delphin.interfaces.ace
- delphin.tdl
- delphin.extra.highlight

# SimpleMrs 1.0

```
[ LTOP: h0
  INDEX: e2 [ e SF: prop-or-ques E.ASPECT: perfective ]
  RELS: < [ "_狗_n_1_rel"<0:1> LBL: h4 ARG0: x3 ]
          [ generic_q_rel<-1:-1> LBL: h5 ARG0: x3 RSTR: h6 BODY: h7 ]
          [ "_叫_v_3_rel"<2:3> LBL: h1 ARG0: e2 ARG1: x3 ARG2: x8 ] >
  HCONS: < h6 qeq h4 h0 qeq h1 > ]
```

# SimpleMrs 1.1

```
[ <0:5> "狗 叫 了"
  TOP: h0
  INDEX: e2 [ e SF: prop-or-ques E.ASPECT: perfective ]
  RELS: < [ "_狗_n_1_rel"<0:1> "狗" LBL: h4 ARG0: x3 ]
          [ generic_q_rel<-1:-1> LBL: h5 ARG0: x3 RSTR: h6 BODY: h7 ]
          [ "_叫_v_3_rel"<2:3> "叫" LBL: h1 ARG0: e2 ARG1: x3 ARG2: x8 ] >
  HCONS: < h6 qeq h4 h0 qeq h1 >
  ICONS: < e2 non-focus x8 > ]
```

# MrsPath improvements

## Reading paths

```
>>> p1 = mp.read_path('"_bark_v_1_rel"[e & @TENSE=past]:ARG1/NEQ>"_dog_n_1_rel"'
>>> p2 = mp.read_path('"_dog_n_1_rel"<ARG1/EQ:"_large_a_1_rel"')
```

## Path exploring and printing with flags

```
>>> mp.format(p1)
'"_bark_v_1_rel"[e & @TENSE=past]:ARG1/NEQ>"_dog_n_1_rel"'
>>> mp.format(p1, flags=mp.PRED|mp.OUT|mp.SUBPATHS)
'"_bark_v_1_rel":ARG1/NEQ>"_dog_n_1_rel"'
>>> mp.format(p1, flags=mp.PRED|mp.OUT)
'"_bark_v_1_rel":ARG1/NEQ>'
>>> mp.format(p1, flags=mp.PRED)
'"_bark_v_1_rel"'
```

# MrsPath improvements

## Merging paths

```
>>> p = mp.merge(p1, p2, location=[':ARG1/NEQ>'])
```

## Reifying MRSs from paths

```
>>> x = next(mp.reify_xmrs(p))
>>> print(simplemrs.dumps_one(x, pretty_print=True))
[ TOP: h0
  RELS: < [ "_bark_v_1_rel" LBL: h2 ARG0: e3 [ e TENSE: past ] ARG1: u4 ]
          [ "_dog_n_1_rel" LBL: h1 ARG0: u4 ]
          [ "_large_a_1_rel" LBL: h1 ARG0: u5 ARG1: u4 ] >
  HCONS: < h0 qeq h2 > ]
```

# ACE interface

```
>>> from delphin.interfaces import ace
>>> with ace.AceParser('/home/goodmami/erg-1212-x86-64-0.9.20.dat') as parser:
...   result = parser.interact('Cats sleep.')
...   print(result.keys())
...   print(result['RESULTS'][0]['MRS'])
...
dict_keys(['NOTES', 'SENT', 'RESULTS', 'ERRORS', 'WARNINGS'])
[ LTOP: h0 INDEX: e2 [ e SF: prop TENSE: pres MOOD: indicative PROG: - PERF: - ]
```

- ace.AceParser
- ace.AceGenerator
- ace.compile()
- ace.parse()
- ace.parse_from_iterable()
- ace.generate()
- ace.generate_from_iterable()

# Basic TDL support

```
>>> from delphin import tdl
>>> defs = list(tdl.parse(open('/home/goodmami/logon/lingo/erg/fundamentals.tdl'
>>> for t in defs[0:5]:
...     print(t.identifier, t.supertypes)
...
sign_min ['*avm*']
sign ['sign_min']
phrase_or_lexrule ['sign']
word_or_lexrule_min ['sign_min']
word_or_lexrule ['word_or_lexrule_min', 'sign']
```

# Other improvements

- Python 2 support
- significant performance improvements
- more unit test coverage
- more documentation
- more contributors!

# gTest Tutorial

gTest is a very simple program for testing grammars.

It currently does the following tasks:

- Regression testing against a gold [incr tsdb()] profile
- Coverage testing on a [incr tsdb()] skeleton

It is designed to be easy to use with conventional grammar configurations.

# gTest basic usage

conventional defaults, simple configuration

- `gTest [GLOBAL-OPTIONS] COMMAND [COMMAND-OPTIONS]`
- `-G` selects the grammar directory
- `R` regression test
- `C` coverage test

E.g. test coverage of all skeletons:

```
$ ./gTest -G ~/repos/zhong/cmn/zhs/ C
```

Test coverage of given skeletons:

```
$ ./gTest -G ~/repos/zhong/cmn/zhs/ C ~/repos/zhong/cmn/zhs/tsdb/interjections
```

# Global configuration

- `-W` working directory (where to store results)
- `-A` ACE config file
- `-C` pre-compiled grammar binary
- `-P` preprocessing command
- `--ace-opts` additional ACE options

```
$ ./gTest -G ~/repos/zhong/cmn/zhs/ \
          -A ~/repos/zhong/cmn/zhs/ace/config-robust.tdl \
          --ace-opts '--max-chart-megabytes=2048'
```

# Colon-ized paths

Once the grammar directory has been set, the other directories are easier to select.

If a colon `:` is prefixed to a path, the path is considered relative to a useful location (e.g. the grammar directory for settings, the skeletons directory for tests). Compare:

```
$ ./gTest -G ~/repos/zhong/cmn/zhs/ \
          -A ~/repos/zhong/cmn/zhs/ace/config-robust.tdl \
          C ~/repos/zhong/cmn/zhs/tsdb/skeletons/mrs/
```

and

```
$ ./gTest -G ~/repos/zhong/cmn/zhs/ -A :ace/config-robust.tdl C :mrs
```

# Regression Testing

list available tests

```
$ ./gTest -G ~/repos/zhong/cmn/zhs/ R --list-profiles
:interjections  /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/interjections
:le     /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/le
:mrs    /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/mrs
:symbols        /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/symbols
:quantifiers    /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/quantifiers
:vv-compounds   /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/vv-compounds
:chart-mapping  /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/chart-mapping
:raising-control        /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/raisir
:tiniest        /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/tiniest
:classifiers    /home/goodmami/repos/zhong/cmn/zhs/tsdb/skeletons/classifiers
```

run a test

```
$ ./gTest -G ~/repos/zhong/cmn/zhs/ R :quantifiers
fail    :quantifiers; See /tmp/tmpzkn8nrvi/run-quantifiers.log
```

# Coverage Testing

```
$ ./gTest -G ~/repos/zhong/cmn/zhs C :le
:le (134 items; 0 ignored):
              :       grammatical        :       ungrammatical
  items       :    38/134   (0.2836)     :    96/134   (0.7164)
  parses      :    35/38    (0.9211)     :    41/96    (0.4271)
  readings    :   150/35    (4.2857)     :   287/41    (7.0000)
```

# gTest vs [incr tsdb()]

- [incr tsdb()] does a lot more
- but gTest doesn't require the LOGON distribution
- gTest is meant for running simple tests at the command line, where [incr tsdb()] is better for active grammar development and tracking history